UD01: SERVICIOS COMPUTACIONALES BASADOS EN MÁQUINAS VIRTUALES

TAREA 1.2: WORDPRESS EN DOS CAPAS

En esta tarea desplegarás **WordPress** separando la aplicación y los datos en dos instancias EC2: una **capa web** con Apache y PHP, accesible desde Internet a través de una **IP elástica**, y una **capa de datos** con **MariaDB**, a la que solo puede llegar la capa web. Es la arquitectura mínima de cualquier aplicación en dos capas y la base sobre la que, más adelante, se añaden balanceadores, autoescalado o bases de datos administradas.

Esta vez lo harás todo **a mano**: crearás los recursos desde la **consola gráfica de AWS** y administrarás las instancias por **SSH desde tu equipo**, usando la instancia web como **host de salto** (bastión) para llegar a la de base de datos. Fíjate en cuántos pasos repetitivos das: son justo los que tendremos que automatizar en una futura tarea.

Tipo: tarea individual evaluable, que se empieza en clase. **Duración estimada:** una sesión en el aula; lo que no termines, acábalo en casa antes de la fecha de entrega.

> **Nota:** la interfaz de AWS Academy y de la consola de AWS cambia con frecuencia, por lo que los menús de **Grupos de seguridad**, **Direcciones IP elásticas** o del asistente de **Lanzar instancia** pueden no coincidir exactamente con lo que se describe aquí. Los nombres de las opciones pueden variar ligeramente, pero los conceptos son los mismos.

OBJETIVOS

- Desplegar una aplicación en dos capas dentro de una VPC, eligiendo región, zona de disponibilidad y subred.
- Restringir el tráfico entre capas con grupos de seguridad que hacen referencia a otros grupos.
- Distinguir cuándo hace falta una IP elástica y cuándo una IP privada fija.
- Administrar una instancia sin acceso SSH desde Internet a través de un host de salto.
- Instalar un servidor web con PHP y conectarlo a un servidor MariaDB remoto aplicando el mínimo privilegio.
- Etiquetar los recursos por proyecto, propietario y rol, y eliminarlos al terminar.

PARTE 1: ARQUITECTURA

Este es el despliegue que vas a construir:

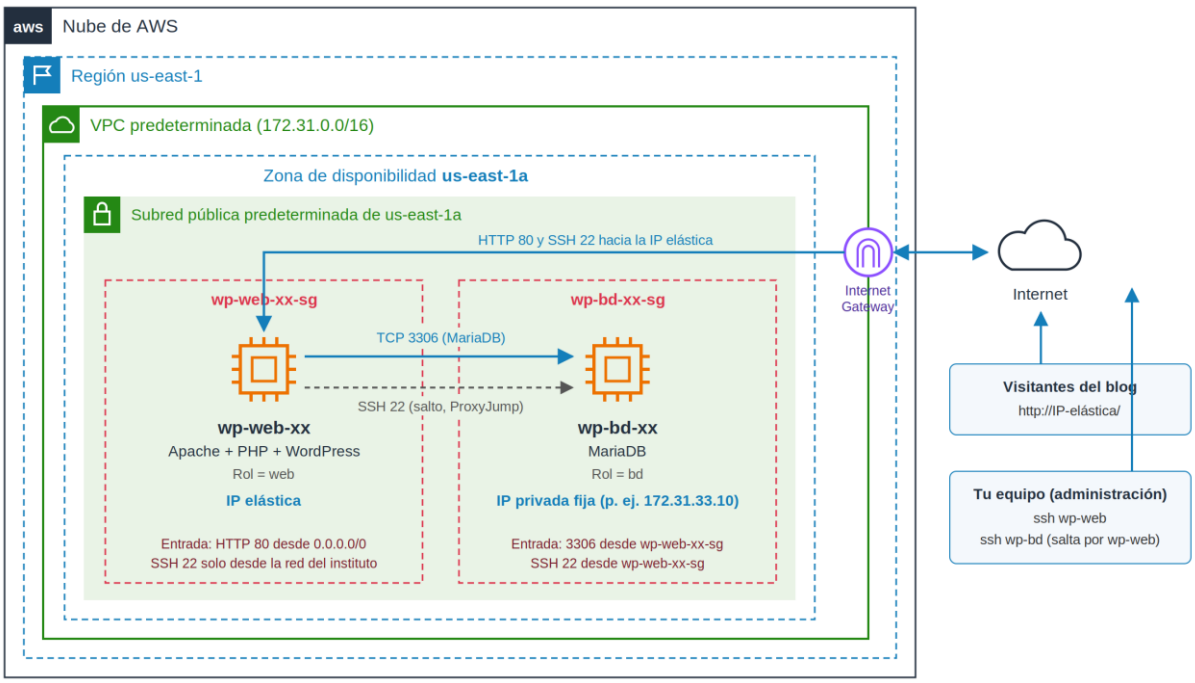


Figura 1. WordPress en dos capas en la subred predeterminada de us-east-1a.

Las dos instancias están en la **misma subred** de us-east-1a. **wp-web-xx** recibe el tráfico HTTP de Internet y el SSH de tu equipo a través de su IP elástica. **wp-bd-xx** también tiene IP pública (la necesita para descargar paquetes), pero su grupo de seguridad solo acepta conexiones al puerto 3306 (MariaDB) y al 22 (SSH) que procedan de instancias del grupo **wp-web-xx-sg**. Desde Internet, por tanto, es inalcanzable.

En los nombres de los recursos, **xx** son tus iniciales en minúsculas: por ejemplo, Ana Pérez Gil usará wp-web-apg, wp-bd-apg, wp-web-apg-sg y wp-bd-apg-sg. El sitio de WordPress se titulará **Blog de Nombre Apellidos**, con tus datos.

Las **contraseñas** (de MariaDB y del administrador de WordPress) las eliges tú: al menos 16 caracteres y distintas entre sí. No deben aparecer en ninguna captura.

> **Importante:** al terminar la sesión de clase deberías tener, como mínimo, las dos instancias en marcha y MariaDB accesible desde la capa web (hasta la Parte 6). Lo ideal es dejar WordPress instalado. Si sigues en casa, tendrás que permitir el SSH desde tu IP (consulta la Parte 3).

PARTE 2: PREPARACIÓN

Inicia el laboratorio y, cuando el indicador esté en verde, abre la consola en la región **us-east-1**. En **AWS Details**, descarga la clave privada de **vockey** (**Download PEM**) y guárdala en la carpeta **.ssh** de tu directorio personal como **labsuser.pem**. SSH rechaza las claves que pueden leer otros usuarios, así que restringe sus permisos:

```
# Tu equipo (Linux o macOS)
chmod 400 ~/.ssh/labsuser.pem

# Tu equipo (Windows, PowerShell)
icacls $HOME\.ssh\labsuser.pem /inheritance:r /grant:r "$($env:USERNAME):R"
```

Elige la IP de la base de datos. Abre **VPC > Subredes**, localiza la subred predeterminada de **us-east-1a** (columna **Zona de disponibilidad**) y anota su **ID** y su **CIDR IPv4**. La numeración de las subredes predeterminadas cambia de una cuenta a otra, por eso no podemos darte una IP fija para todos. Elige la tuya sumando 1 al tercer octeto de la dirección de red y usando 10 como último: por ejemplo, si el CIDR es 172.31.32.0/20, la IP será **172.31.33.10**. Así quedas lejos de las cuatro primeras direcciones y de la última, que AWS reserva en cada subred.

Los comandos de esta tarea se ejecutan en tres sitios distintos. Cada bloque indica en su primera línea dónde se ejecuta y **todo su texto lleva un color**: **amarillo** para wp-web-xx, **verde** para wp-bd-xx y **azul claro** para el **terminal de tu equipo**.

PARTE 3: GRUPOS DE SEGURIDAD

Los crearemos **antes** que las instancias, porque las reglas de wp-bd-xx-sg hacen referencia a wp-web-xx-sg, que tiene que existir. En **EC2 > Red y seguridad > Grupos de seguridad**, elige **Crear grupo de seguridad** dos veces, en la VPC predeterminada, con las etiquetas **Name**, **Proyecto** = UD01, **Propietario** = xx y **Rol** = web o bd:

Grupo	Tipo	Puerto	Origen
wp-web-xx-sg	HTTP	80	Anywhere-IPv4 (0.0.0.0/0)
wp-web-xx-sg	SSH	22	Personalizado: rango del instituto
wp-bd-xx-sg	MYSQL/Aurora	3306	Personalizado: wp-web-xx-sg
wp-bd-xx-sg	SSH	22	Personalizado: wp-web-xx-sg

Deja las reglas de salida como están (todo el tráfico permitido). En el origen de las reglas de wp-bd-xx-sg, empieza a escribir **sg-** o el nombre del grupo y elígelo de la lista.

> **Importante:** desde el instituto **no sirve la opción Mi IP**, porque la red del centro sale a Internet por varias direcciones públicas: usa el **rango CIDR** que te daremos en clase. Si trabajas desde casa, añade otra regla SSH con origen **Mi IP** y bórrala al terminar. No abras nunca el puerto 22 a 0.0.0.0/0.
>
> **Para pensar:** ¿qué ventaja tiene usar como origen el grupo wp-web-xx-sg en lugar de la IP privada de wp-web-xx? Piensa en qué pasaría si sustituyeras la instancia web o añadieras una segunda.

PARTE 4: INSTANCIAS E IP ELÁSTICA

Lanza las dos instancias desde **Instancias > Lanzar instancias**. En **Configuración de red** pulsa **Editar** para elegir la subred, la IP pública y el grupo existente; la IP principal está en **Configuración de red avanzada** y el perfil IAM, en **Detalles avanzados**. Como el cifrado de EBS predeterminado sigue activo en la cuenta, los volúmenes se crearán cifrados.

Campo	wp-web-xx	wp-bd-xx
Etiquetas (instancias y volúmenes)	Name, Proyecto = UD01, Propietario = xx, Rol = web	Name, Proyecto = UD01, Propietario = xx, Rol = bd
AMI	Ubuntu Server 26.04 LTS (x86)	Ubuntu Server 26.04 LTS (x86)
Tipo de instancia	t3.micro	t3.micro
Par de claves	vockey	vockey
Subred	Predeterminada de us-east-1a	Predeterminada de us-east-1a
Asignar IP pública automáticamente	Habilitar	Habilitar
Grupo de seguridad	Existente: wp-web-xx-sg	Existente: wp-bd-xx-sg
IP principal	(automática)	La que elegiste en la Parte 2
Almacenamiento	10 GiB gp3	10 GiB gp3
Perfil de instancia de IAM	LabInstanceProfile	LabInstanceProfile

Asigna la IP elástica. En **Red y seguridad > Direcciones IP elásticas**, elige **Asignar la dirección IP elástica**, añade las etiquetas (Name = wp-web-xx-eip, Proyecto, Propietario y Rol = web) y asígnala. Después, con ella seleccionada, elige **Acciones > Asociar la dirección IP elástica** y escoge wp-web-xx. En los detalles de la instancia, comprueba que su **IPv4 pública** es ahora la elástica.

¿Por qué la necesitamos? **WordPress guarda su propia URL en la base de datos** durante la instalación. Si la IP pública de la instancia web cambiara al detenerla y arrancarla, el sitio redirigiría a una dirección que ya no existe. Por eso conviene asociar la IP elástica **antes** de instalar WordPress. La IP privada, en cambio, no cambia al detener y arrancar una instancia: fijarla en wp-bd-xx sirve para conocerla antes del lanzamiento y poder reutilizarla si un día reconstruyes la instancia (por ejemplo, desde una AMI) sin tocar la configuración de WordPress.

> **Para pensar:** wp-bd-xx tiene IP pública, pero nadie puede conectarse a ella desde Internet. ¿Qué elemento lo impide? ¿Por qué no basta con quitarle la IP pública para que siga funcionando todo igual?

PARTE 5: ACCESO SSH CON HOST DE SALTO

En lugar de copiar la clave privada a wp-web-xx o de reenviar el agente SSH (ssh -A), usaremos **ProxyJump**: tu equipo abre un túnel a través de wp-web-xx y negocia directamente la conexión con wp-bd-xx. La clave **nunca sale de tu equipo**. Crea (o amplía) el fichero **~/.ssh/config** de tu equipo (en Windows, **C:\Users\tu_usuario\.ssh\config**, sin extensión) con estas dos entradas, sustituyendo los valores entre < >:

```
# Tu equipo: contenido de ~/.ssh/config
Host wp-web
    HostName <IP elástica de wp-web-xx>
    User ubuntu
    IdentityFile ~/.ssh/labsuser.pem

Host wp-bd
    HostName <IP privada de wp-bd-xx>
    User ubuntu
    IdentityFile ~/.ssh/labsuser.pem
    ProxyJump wp-web
```

Pruébalo. La primera vez tendrás que aceptar la huella de cada servidor:

```
# Tu equipo
ssh wp-web hostname -I
ssh wp-bd hostname -I
```

Cada orden devuelve la IP privada de la instancia correspondiente. Ahora intenta entrar directamente en wp-bd-xx por su IP pública: la conexión se quedará esperando hasta agotar el tiempo, porque el grupo de seguridad descarta el paquete.

```
# Tu equipo
ssh -o ConnectTimeout=10 -i ~/.ssh/labsuser.pem \
    ubuntu@<IP pública de wp-bd-xx>
```

A partir de aquí, abre dos terminales en tu equipo: uno con **ssh wp-web** y otro con **ssh wp-bd**.

PARTE 6: CAPA DE DATOS CON MARIADB

Instala MariaDB y comprueba en qué dirección escucha:

```
# wp-bd-xx
sudo apt update && sudo apt install -y mariadb-server
systemctl status mariadb --no-pager
sudo ss -tlnp | grep 3306
```

Por defecto solo escucha en **127.0.0.1**: acepta conexiones locales, no las de wp-web-xx. En lugar de editar el fichero de configuración del paquete, añade uno propio en el directorio de configuración, que se lee después y se conserva en las actualizaciones:

```
# wp-bd-xx
sudo tee /etc/mysql/mariadb.conf.d/99-wordpress.cnf > /dev/null << 'EOF'
[mysqld]
bind-address = 0.0.0.0
EOF
sudo systemctl restart mariadb
sudo ss -tlnp | grep 3306
```

Ahora escucha en todas las interfaces. Crea la base de datos **wordpress** y su usuario **wpuser**. El usuario solo podrá conectarse **desde la IP privada de wp-web-xx** (consúltala en la consola o con hostname -I) y solo tendrá permisos sobre su base de datos:

```
# wp-bd-xx (dentro de sudo mariadb)
CREATE DATABASE wordpress CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wpuser'@'<IP privada de wp-web-xx>' IDENTIFIED BY '<contraseña>';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'<IP privada de wp-web-xx>';
SELECT user, host FROM mysql.user;
SHOW GRANTS FOR 'wpuser'@'<IP privada de wp-web-xx>';
EXIT;
```

PARTE 7: CAPA WEB CON APACHE, PHP Y WORDPRESS

Instala Apache, PHP con las extensiones que necesita WordPress y el cliente de MariaDB, que usaremos para comprobar la conexión **antes** de instalar nada más:

```
# wp-web-xx
sudo apt update
sudo apt install -y apache2 libapache2-mod-php php-mysql php-curl php-gd \
     php-intl php-mbstring php-xml php-zip php-imagick mariadb-client
mariadb -h <IP privada de wp-bd-xx> -u wpuser -p \
    -e "SELECT USER(), @@hostname;"
```

La consulta debe mostrar **wpuser@<IP privada de wp-web-xx>** (así te ve MariaDB) y el nombre de host de wp-bd-xx. Si falla, no sigas: revisa la tabla del final de esta parte.

Descarga WordPress en español y dáselo al usuario de Apache:

```
# wp-web-xx
cd /tmp && wget https://es.wordpress.org/latest-es_ES.tar.gz
sudo tar xf latest-es_ES.tar.gz -C /var/www
sudo chown -R www-data:www-data /var/www/wordpress
```

Crea un sitio de Apache que sirva WordPress en la raíz (http://IP-elástica/), permita sus ficheros .htaccess y sustituya al sitio por defecto:

```
# wp-web-xx
sudo tee /etc/apache2/sites-available/wordpress.conf > /dev/null << 'EOF'
<VirtualHost *:80>
    DocumentRoot /var/www/wordpress
    <Directory /var/www/wordpress>
        AllowOverride All
        Require all granted
    </Directory>
    ErrorLog ${APACHE_LOG_DIR}/wordpress_error.log
    CustomLog ${APACHE_LOG_DIR}/wordpress_access.log combined
</VirtualHost>
EOF
sudo a2enmod rewrite
sudo a2dissite 000-default && sudo a2ensite wordpress
sudo apache2ctl configtest && sudo systemctl restart apache2
```

Instala WordPress desde el navegador en **http://<IP elástica>/**. En la pantalla de la base de datos, indica el nombre **wordpress**, el usuario **wpuser**, su contraseña, el servidor **<IP privada de wp-bd-xx>** y deja el prefijo **wp_**. En la siguiente, el título **Blog de Nombre Apellidos**, un usuario administrador que no se llame «admin» y una contraseña robusta.

Si algo no funciona:

Síntoma	Qué revisar
ssh wp-web: Connection timed out	La instancia está detenida o te conectas desde una red que no está en las reglas SSH de wp-web-xx-sg (el rango del instituto o, desde casa, Mi IP).
UNPROTECTED PRIVATE KEY FILE	Los permisos de labsuser.pem son demasiado abiertos. Repite chmod o icacls (Parte 2).
ssh wp-web funciona, pero ssh wp-bd no	La regla SSH de wp-bd-xx-sg no tiene como origen wp-web-xx-sg, o la IP de HostName es incorrecta.

Síntoma	Qué revisar
ERROR 2002 Can't connect ... (111)	Conexión rechazada: MariaDB solo escucha en 127.0.0.1. Revisa el fichero 99-wordpress.cnf y reinicia.
ERROR 2002 Can't connect ... (110 o 115)	Tiempo agotado: el grupo de seguridad no deja pasar el 3306. Revisa la regla de wp-bd-xx-sg.
ERROR 1045 Access denied	El host del usuario no es la IP privada de wp-web-xx o la contraseña no coincide. Revisa SELECT user, host.
Error al establecer una conexión con la base de datos	Las mismas causas. Prueba antes con el cliente mariadb desde wp-web-xx.
Aparece la página por defecto de Apache	El sitio 000-default sigue activo. Revisa a2dissite y reinicia Apache.
WordPress redirige a otra IP	Lo instalaste antes de asociar la IP elástica. Corrige siteurl y home en la tabla wp_options.

PARTE 8: PRIMERA ENTRADA Y COMPROBACIONES

Publica una entrada que incluya una imagen: por ejemplo, una captura de tu consola de EC2 con las dos instancias. Después, busca dónde ha quedado cada cosa:

```
# wp-web-xx
find /var/www/wordpress/wp-content/uploads -type f

# wp-bd-xx
sudo mariadb wordpress -e "SELECT ID, post_type, post_title FROM wp_posts \
  WHERE post_status IN ('publish','inherit');"
```

El texto de la entrada está en la base de datos, pero la imagen está en el disco de wp-web-xx. Tenlo en cuenta: cuando haya varios servidores web, los ficheros subidos tendrán que estar en un almacenamiento compartido.

Comprueba la seguridad desde fuera. Desde tu equipo, la web responde y el puerto 3306 de wp-bd-xx no (el comando debe fallar por tiempo agotado):

```
# Tu equipo (Linux o macOS)
curl -sI http://<IP elástica>/ | head -1
nc -zv -w 5 <IP pública de wp-bd-xx> 3306

# Tu equipo (Windows, PowerShell)
(Invoke-WebRequest http://<IP elástica>/ -Method Head).StatusCode
Test-NetConnection <IP pública de wp-bd-xx> -Port 3306
```

PARTE 9: AMPLIACIÓN OPCIONAL: BASE DE DATOS EN UNA SUBRED PRIVADA

Intenta esta parte si en el módulo de redes ya has trabajado las **subredes privadas** y las **tablas de rutas**. En empresa, la capa de datos no tendría IP pública ni ruta hacia Internet. Lleva wp-bd-xx a una **subred privada** sin reconstruirla desde cero:

1. **Subred.** En **VPC > Subredes**, crea **wp-priv-xx** en la VPC predeterminada, zona us-east-1a, con el CIDR **172.31.96.0/24** (las subredes predeterminadas ocupan hasta 172.31.95.255).
2. **Tabla de rutas.** Crea **wp-priv-xx-rt**, que solo tendrá la ruta local, y asócíala a wp-priv-xx. Una subred es privada porque su tabla de rutas no lleva a la puerta de enlace de Internet; si no la asocias, usará la tabla principal de la VPC, que sí lo hace.
3. **AMI.** Crea una imagen de wp-bd-xx (**Acciones > Imagen y plantillas > Crear imagen**) llamada wp-bd-xx-ami, con sus etiquetas.
4. **Nueva instancia.** Lanza **wp-bdpriv-xx** desde la AMI en wp-priv-xx, sin IP pública, con wp-bd-xx-sg, la IP privada 172.31.96.10 y las mismas etiquetas que wp-bd-xx (Rol = bd).
5. **Reconfigura.** Cambia DB_HOST en /var/www/wordpress/wp-config.php y HostName en la entrada wp-bd de ~/.ssh/config. No hay que tocar nada más: las direcciones que WordPress guarda en la tabla wp_options (siteurl y home) son las de la capa web, que no cambia, y el usuario de MariaDB sigue siendo válido porque la IP privada de wp-web-xx es la misma. Comprueba que el blog funciona y termina wp-bd-xx.

Sin ruta a Internet, wp-bdpriv-xx no puede instalar actualizaciones. Si quieres probarlo, crea una **puerta de enlace NAT** pública en la subred predeterminada de us-east-1a, añade a wp-priv-xx-rt la ruta 0.0.0.0/0 hacia ella y ejecuta sudo apt update en wp-bdpriv-xx.

> **Importante:** la puerta de enlace NAT se factura por hora y por GB procesado, y consume presupuesto del laboratorio aunque no la uses. Elimínala en cuanto termines la prueba y libera después su IP elástica.

PARTE 10: LIMPIEZA Y FIN DE LA TAREA

Haz la limpieza **cuando tengas todas las capturas de la entrega**. Hasta entonces, detén las instancias al terminar cada día: la IP elástica sigue asociada y, al arrancarlas, todo vuelve a funcionar.

1. **Instancias.** Termina wp-web-xx y wp-bd-xx (y wp-bdpriv-xx, si hiciste la ampliación). Sus volúmenes raíz se eliminan con ellas.
2. **IP elástica.** Cuando wp-web-xx esté terminada, la dirección queda sin asociar y se sigue facturando. Selecciónala y elige **Acciones > Liberar direcciones IP elásticas**.

3. **Grupos de seguridad.** Elimina primero wp-bd-xx-sg: mientras sus reglas hagan referencia a wp-web-xx-sg, AWS no te dejará borrar este último.
4. **Ampliación.** Si la hiciste, anula el registro de la AMI, elimina su snapshot, la puerta de enlace NAT y su IP elástica, la tabla de rutas y la subred.

Comprueba el resultado con la CLI. Como todos los recursos llevan la etiqueta Rol, basta un filtro:

```
# Tu equipo (AWS CLI)
Q="Reservations[].Instances[].[Tags[?Key=='Name']|[0].Value,State.Name]"
aws ec2 describe-instances --output table --query "$Q" \
  --filters "Name=tag:Rol,Values=web,bd" "Name=tag:Propietario,Values=xx"
aws ec2 describe-addresses --query "Addresses[].[PublicIp,AssociationId]"
aws ec2 describe-security-groups --filters "Name=tag:Rol,Values=web,bd" \
  --query "SecurityGroups[].GroupName"
```

Las instancias deben aparecer como **terminated** y las otras dos listas, vacías. No borres la plantilla de lanzamiento ni los grupos de seguridad de otras prácticas: se usan más adelante. Para terminar, pulsa **End Lab**.

ENTREGA

Entrega en el aula virtual un único PDF llamado **CE_RSN-ARCN-UD01-Task1_2-Apellidos_Nombre.pdf** con las siguientes capturas, cada una con un breve comentario:

1. **Instancias** en la consola de EC2, en ejecución, con las columnas Name, ID, zona, IP privada, IP pública y grupos de seguridad.
2. **Reglas de entrada** de wp-web-xx-sg y wp-bd-xx-sg, y la **IP elástica** asociada a wp-web-xx, con sus etiquetas.
3. **Terminal de tu equipo** con el contenido de ~/.ssh/config, la salida de ssh wp-bd hostname -I y el intento fallido de conectar al puerto 3306 de wp-bd-xx.
4. **wp-bd-xx** con la salida de ss -tlnp | grep 3306, SELECT user, host y SHOW GRANTS (sin contraseñas).
5. **Navegador** con la página de inicio del blog, con su título personalizado y tu entrada. La barra de direcciones debe mostrar la IP elástica.
6. Si hiciste la ampliación, el **mapa de recursos** de la VPC y el blog funcionando con la base de datos en la subred privada.

Añade al final tus respuestas a las preguntas **Para pensar** (entre dos y cinco líneas cada una).

RESUMEN DE OPERACIONES

Operación	Consola	AWS CLI
Crear un grupo de seguridad	Grupos de seguridad > Crear grupo de seguridad	`aws ec2 create-security-group`
Regla con otro grupo como origen	Editar reglas de entrada > Origen: Personalizado	`aws ec2 authorize-security-group-ingress --source-group`
IP privada fija	Lanzar instancia > Configuración de red avanzada > IP principal	`aws ec2 run-instances --private-ip-address`
Asignar y asociar una IP elástica	Direcciones IP elásticas > Asignar / Acciones > Asociar	`aws ec2 allocate-address / associate-address`
Liberar una IP elástica	Acciones > Liberar direcciones IP elásticas	`aws ec2 release-address`

Comando o fichero	Para qué sirve
`~/.ssh/config con ProxyJump`	Saltar a una instancia interna a través de un bastión sin copiar la clave.
`ss -tlnp`	Ver qué puertos escuchan y en qué dirección.
`mariadb.conf.d/99-*.cnf`	Cambiar la configuración de MariaDB sin tocar los ficheros del paquete.
`CREATE USER 'u'@'IP'`	Limitar desde qué equipo puede conectarse un usuario de MariaDB.
`mariadb -h IP -u usuario -p`	Probar la conexión remota antes de configurar la aplicación.
`a2ensite / a2dissite / apache2ctl configtest`	Activar y desactivar sitios de Apache y validar su configuración.